PYTHON LISTS

1. List Basics:

A list in Python is a collection of items, ordered and mutable. It can contain elements of different data types, including other lists. Lists are defined using square brackets `[]`. You can perform various operations on lists, such as accessing individual elements, slicing, appending, and more.

Example:

```
# Creating a list
my_list = [1, 2, 3, 'apple', 'banana']

# Accessing elements
print(my_list[0])    # Output: 1
print(my_list[3])    # Output: apple

# Modifying elements
my_list[1] = 'orange'
print(my_list)       # Output: [1, 'orange', 3, 'apple', 'b
anana']
```

1. Indexing:

In Python, indexing refers to the process of accessing individual elements in a data structure, such as a list, tuple, or string, using their position or index within the data structure. The index is a numerical value that represents the position of an element in the sequence, and it usually starts from 0.

```
my_list = [10, 20, 30, 40, 50]

# Accessing individual elements
first_element = my_list[0]   # 10
third_element = my_list[2]   # 30
last_element = my_list[-1]   # 50
```

```
print(first_element, third_element, last_element)
```

2. Slicing:

In Python, slicing is a way to extract a portion or a subsequence of elements from a sequence (like a list, tuple, or string). The syntax for slicing is generally `sequence[start:stop:step]`, where:

- `start`: The index of the first element you want in the slice.
- `stop`: The index of the first element you do not want in the slice (it's an exclusive boundary).
- `step` (optional): The step or interval between elements in the slice. If omitted, it defaults to 1.

```
my_list = [10, 20, 30, 40, 50, 60, 70, 80]

# Slicing to get sublists
sublist1 = my_list[2:4]    # [30, 40]
sublist2 = my_list[4:]     # [50, 60, 70, 80]
sublist3 = my_list[:3]     # [10, 20, 30]

print(sublist1, sublist2, sublist3)
```

3. Using Negative Indexing:

Negative indexing in Python allows you to access elements from the end of a sequence by using negative integers as indices. In other words, with negative indexing, you can count elements from the end of the sequence, where -1 represents the last element, -2 represents the second-to-last element, and so on.

```
my_list = [10, 20, 30, 40, 50]

# Negative indexing to access elements from the end
second_last_element = my_list[-2]   # 40
last_three_elements = my_list[-3:]  # [30, 40, 50]
```

```
print(second_last_element, last_three_elements)
```

4. Skipping Characters (Step):

```
my_list = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Slicing with a step to skip elements
every_second_element = my_list[::2]  # [10, 30, 50, 70, 90]
reverse_list = my_list[::-1]         # [90, 80, 70, 60, 50, 4

print(every_second_element, reverse_list)
```

3. List Methods:

Python provides various built-in methods to manipulate lists. Some common methods include `append()`, `extend()`, `copy()`, `clear()`, `count()`, `index()`, `remove()`, `pop()`, `insert()`, `reverse()` and `sort()`.

Example:

```
# List methods example
numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5]

numbers.append(8)          # Appending an element
numbers.extend([7, 0])     # Extending with another list
numbers.copy()             # Returns the shallow copy of th
e list
numbers.clear()            # Removes all elements from the
list
numbers.count(2)           # Returns the number of occurren
ces of a specified
                                                  # item
in the list.
numbers.index(4)           # Returns the index of the first
occurrence of a
                                                  # spec
ified item in the list.
numbers.remove(1)          # Removing the first occurrence
```

```
of 1
numbers.pop(4)              # Removing element at index 4
numbers.insert(2, 10)       # Inserting 10 at index 2
numbers.reverse()           # Reverses the elements of the l
ist
numbers.sort()              # Sorting the list

print(numbers)              # Output: [2, 3, 4, 5, 6, 7, 8,
9, 10]
```

4. Nested List:

A nested list is a list that contains other lists. This allows for creating a matrix-like structure or representing hierarchical data.

Example:

```
# Nested list example
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# Accessing elements
print(matrix[0][1])    # Output: 2
print(matrix[2][0])    # Output: 7
```

5. List Comprehensions:

List comprehensions provide a concise way to create lists. They consist of an expression followed by a `for` loop inside square brackets.

Example:

```
Syntax
[expression for item in iterable]

# List comprehension example
squares = [x**2 for x in range(5)]

# Equivalent to:
# squares = []
```

```
# for x in range(5):
#     squares.append(x**2)

print(squares)    # Output: [0, 1, 4, 9, 16]
```

List comprehensions can also include conditions:

```
# List comprehension with condition
#[expression for item in iterable if condition]
even_squares = [x**2 for x in range(10) if x % 2 == 0]

print(even_squares)    # Output: [0, 4, 16, 36, 64]
```